UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Distributed Systems

Introduction and basic concepts

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

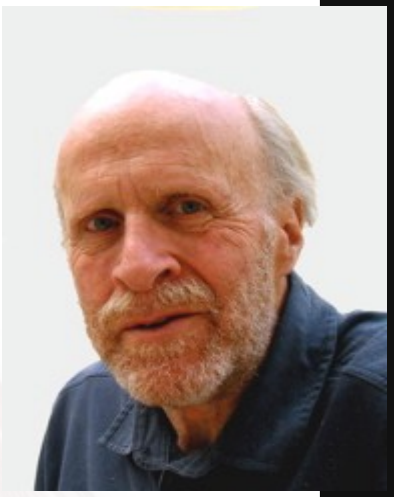
Contents

- Distributed System concepts and examples
- Features, pros and cons.
- Distributed computing vs. Parallel computing
- Nomenclature: protocols, clients, services and servers
- Middleware
- Distributed computing paradigms

Definitions

A **distributed system** is one in which components located at networked computers communicate and coordinate their actions **only** by passing messages.

***G. Coulouris**, J. Dollimore & T. Kindberg (2001)*



The main characteristics of distributed systems:

- Concurrent execution of processes
- Lack of a global clock
- Independent failures

Most of today's applications: social networking, cloud applications, grid computing, mobile computing, IoT, etc. are **distributed systems**.

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Independent failures?

Client and server are independent processes potentially running on different nodes. They may have particular circumstances and restrictions.

Client and server run independently and may disagree about the outcome.

- **Server:** completes the operation and sends a reply, so the call **success**.
- **Client:** the reply arrives after its deadline, so the call **fails**.
- Should the client retry the operation?

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Distributed System

System consists of:

- computer resources (hardware and software)
- physically distributed
- connected through a computer network
- that communicate and coordinate their actions by passing messages and
- cooperate to perform a task, providing an integrated service.

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Distributed System vs. Distributed Application

- The DS is an **infrastructure**: it includes nodes and devices, but also software (middleware and support services). The DA is just “client-facing” **software**.
- The DS is a **resource manager**, it provides support for common features: communications, mutual exclusion, resource location, security, etc. The DA is specific software for a specific purpose. It’s an **application**.
- The DA runs on top a DS. Several DA’s may run on top the same DS.

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

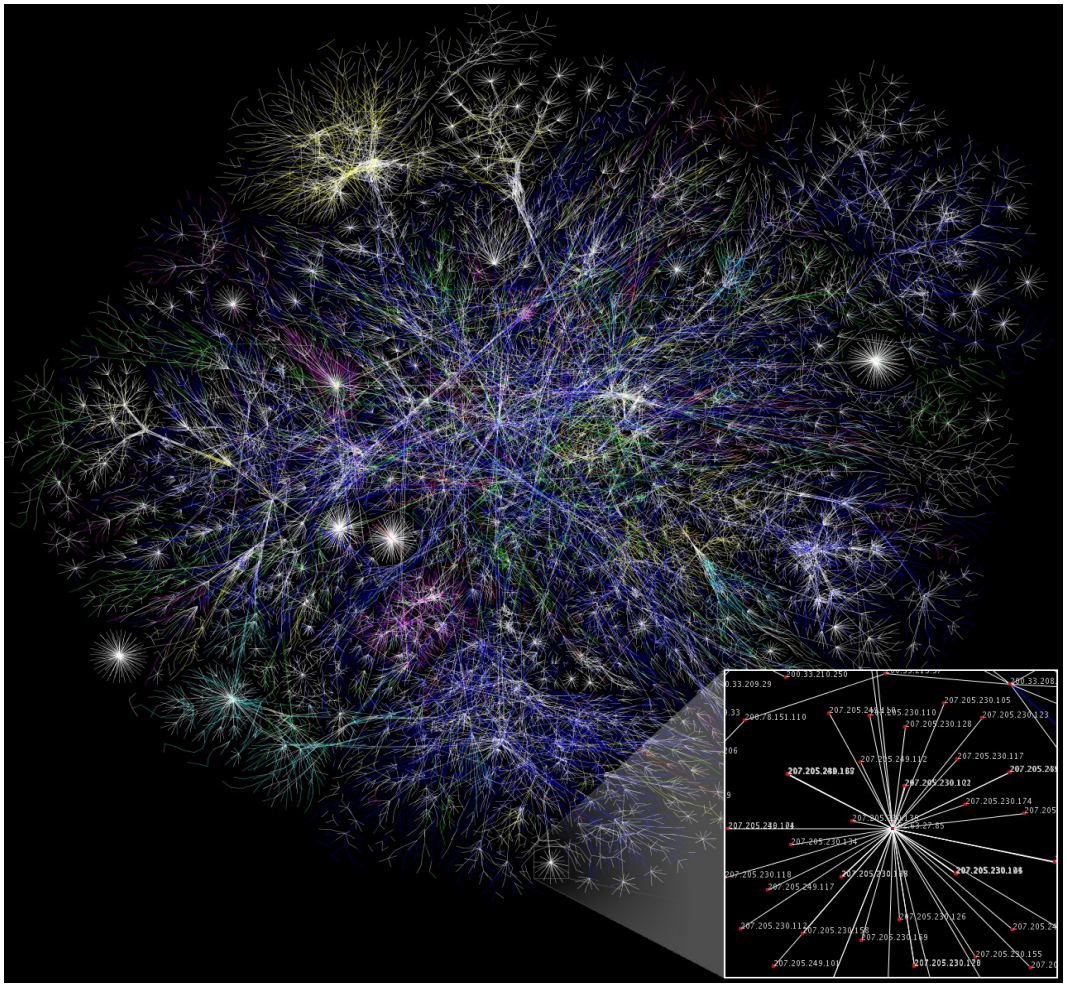
Origin: Resource Sharing

- **Cooperation:** programs on different machines work together.
- **Resource sharing:** programs make local resources available remotely.
- **Protocols:** describe interactions as procedures and calling sequences.
- **Development support:** languages, compilers and runtimes help build distributed applications.

J. E. White, RFC 707 (1976)

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Example
Internet and web search



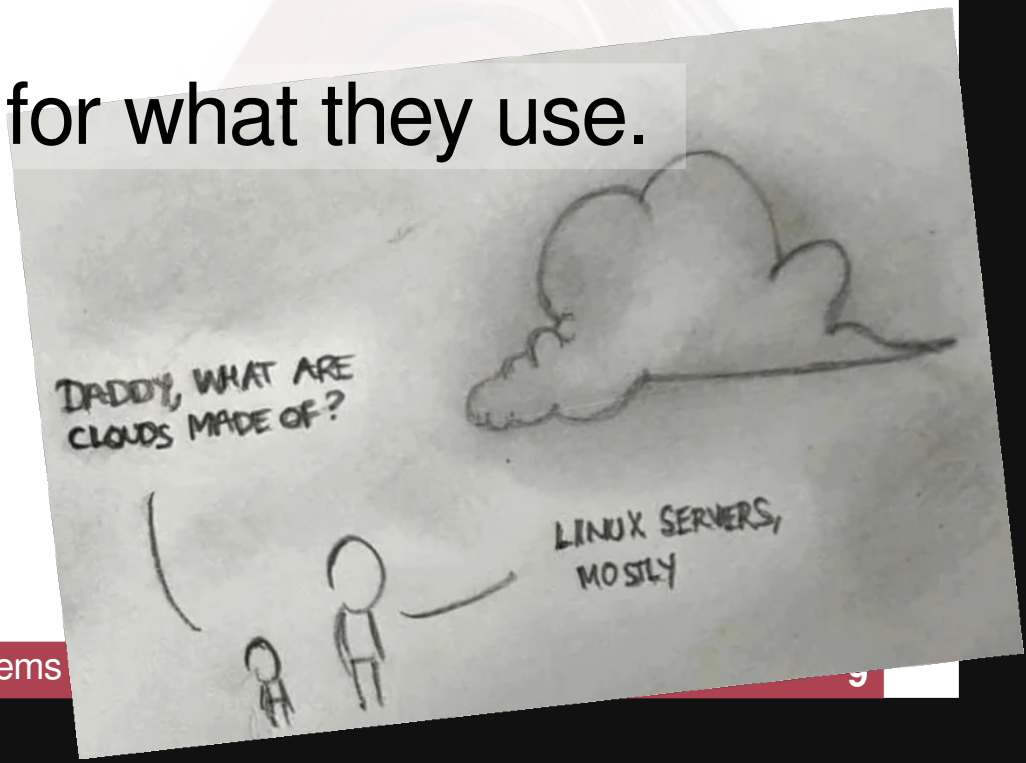
$63 \cdot 10^9$ web pages
$1 \cdot 10^{12}$ URLs

http://www.internetlivestats.com/google-search-statistics/

Example
Cloud Computing

Cloud computing provides arbitrary software and hardware IT resources over the Internet:

- On-demand.
- That scale with the workload
  (they are "elastic" resources).
- The customer pays only for what they use.



UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Example
Cloud Computing types

Infrastructure as a Service (IaaS): It provides IT resources (nodes, storage, networks), but the customer manages them.

- Examples: AWS, Azure, Google Cloud

Platform as a Service (PaaS): The customer is only concerned with the application build. The provider manages the IT resources.

- Examples: Heroku, Google App Engine

Software as a Service (SaaS): Both the infrastructure and the application itself are managed by the provider.

- Examples: Microsoft 365, Slack, Dropbox

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Pros and cons

Pros

- Sharing resources (hardware, software, data)
- Good cost/performance ratio
- Growth capacity (scalability)
- Higher availability (fault tolerance)
- Concurrency: simultaneous service to multiple users
- Speed: increased processing capacity

Cons

- Interconnection of resources (cost, reliability, saturation, etc.)
- Communications security
- More complex software

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Distributed vs. Parallel

Distributed Systems:

- Goal: resource sharing
- Heterogeneous computer networks
  - Set of computers connected by an heterogeneous inter-network

Parallel Systems:

- Goal: high performance (speedup) and high productivity
- Dedicated architectures (parallel machines):
  - Multiprocessors
  - Multicomputers

Nomenclature

- **SERVICE** – a component managing a **collection of related resources** and provides functionality to users and applications
- The only access to the service is via a well-defined **set of operations** that it exports.
  - **Example**: A file server just provides *read*, *write* and *delete* operations.
- For effective sharing, each resource must be managed by a program that offers a communication interface This **communication interface** composed by a **set of operations** is often called **Application Programming Interface (API)**
- The APIs are a key concept in new data-based companies
- Example: Instagram

  https://developers.facebook.com/docs/instagram-basic-display-api



UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Nomenclature

- **SERVER** – A running program (a *process*) on a networked computer that accepts **requests** from another program running on other computer to perform a **service** and **responds** appropriately.
- The requesting processes are referred as clients, and the overall approach is known as client-server computing.
- When a client sends a request for an operation, we say that the client invokes an operation upon the server.
- A **server** can implement/provide one or several **services** using, usually, an API for each service.
- ¿How many servers are running in your laptop now?

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Nomenclature

- A protocol is the **set of rules** that enable two entities to connect and transmit data to one another (message exchange).
- The **set of messages** that a server can accept refers to a concrete protocol.
- A **protocol** specification requires:
  - Syntax
  - Semantics
  - Synchronization / Timing
- Many RFCs contains protocol specifications

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Client and server is a ...

DIY

- program | role
- process | device
- always running | just a while
- passive | active
- invokes | replies
- heavy | light
- simple | complex

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Client or Server?

DIY

- ping
- Chrome (web browser)
- Squid (web proxy)
- IoT device
- Dropbox (desktop)
- *Other?*

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Challenges

- **Heterogeneity** of hardware, software, networks and uses.
- **Openness**: ready to extend, provides open interfaces.
- **Security (CIA)**: Confidentiality, Integrity, Availability
- **Scalability**: Remain effective when system grows (more resources and users)
- **Failure handling**: Ready to manage failures in any process, computer or network.
- **Concurrency**: Resources may be requests concurrently.
- **Transparency**: Show the system as a whole rather than a set of components. Hiding details to users/programmers.
- **Quality of Service**: how the service should be provided:
  - Reliability, security and performance.

Heterogeneity

Variety and difference in its components:

- Networks: topology, technology, configuration, etc.
- Hardware: servers, mobiles, laptops, printers, robots, motors, sensors, etc.
- Operating Systems: Windows, Linux, Android, FreeRTOS, VxWorks, etc.
- Programming languages: Java, C++, Python, C, Lisp, etc.
- Programmers, experience, know-how...
- E.g. KPI: Time of integration of a new technology.

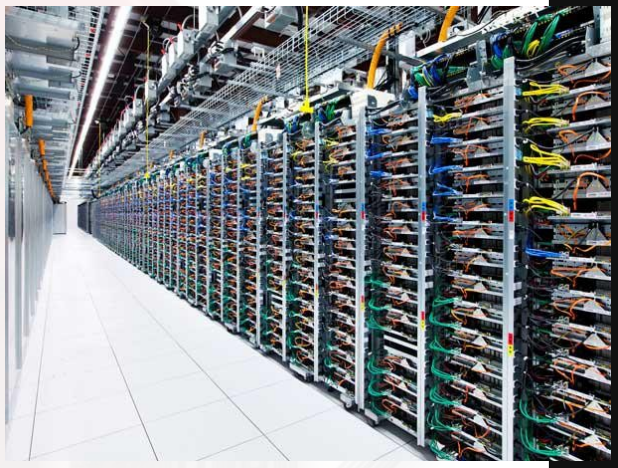
How do we solve heterogeneity?

- Using open systems

Scalability

A system has the capacity to grow (**scalability**) if it maintains its effectiveness when the number of resources or users increases significantly

- A system with *n* users is scalable if the amount of resources needed to support it is proportional to n (O(n))
- Example: Internet growth
- Ex. KPI: Number of users served per minute/hour/.



Aspects to be considered:

- Cost/Performance
- Resource availability (ex. IPv4 addresses)

The concept of **cloud** was born, among other reasons, to provide ad-hoc scalability.

- Resource outsourcing

https://www.crn.com

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Concurrency

- Concurrent processes must communicate and synchronize
- Basic form of communication and synchronization in a distributed system
  - Passing messages
    - send (message, destination)
    - receive (message, source)
  - Synchronous and asynchronous mechanisms
  - Direct or indirect communication
  - Fixed or variable message size
  - Group communication
    - Multicast, broadcast

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Concurrency → **Consistency**

- **Concurrency:** several processes access shared resources at the same time.
- **Coordination:** processes exchange messages to synchronize their actions.
- **Consistency:** define which values readers may observe after updates.
- **Replication and caching:** copies must follow the chosen consistency rules.

Example: two clients update the same file. Which version should readers see?

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Reliability

- **Partial failure:** one component may fail while others keep running.
- **Reliability:** probability of correct operation over a period of time.
- **Availability:** fraction of time the service is usable.
- **Fault tolerance:** detect or mask failures, isolate faults and recover.
- **Redundancy:** provide alternative components or data copies.

Transparency

Hiding details of the system components from the user:

- **Access**: The same interface (operations) to access local and remote resources.
- **Location**: Access is independent of the location of the resource.
- **Concurrency**: Several processes access the same resource concurrently.
- **Replication**: Users are not aware of the existence of replicas.
- **Failure**: Fails are hidden from clients (but not all).
- **Mobility**: of both resources and user without affecting their operations.
- **Performance**: allowing optimization according to the load.
- **Scalability**: it works the same way regardless of the amount of resources or clients.

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

What kind(s) of transparency provides?

DIY

- Spanning Tree Protocol
- e-mail
- RAID (Redundant Array of Independent Disks)
- URL
- DNS
- TCP
- OSPF

Middleware

A middleware is a software infrastructure (which may include tools, libraries, protocols, services, etc.) that aims to facilitate the development of applications in distributed environments.

- It allows masking the heterogeneity of the underlying networks:
  - Supports various programming models, programming languages, etc.
  - It supports various operating systems
- Provide a common programming model for developers
- They provide common structural services to all applications.
- Naming, security, persistence, transactions, etc.





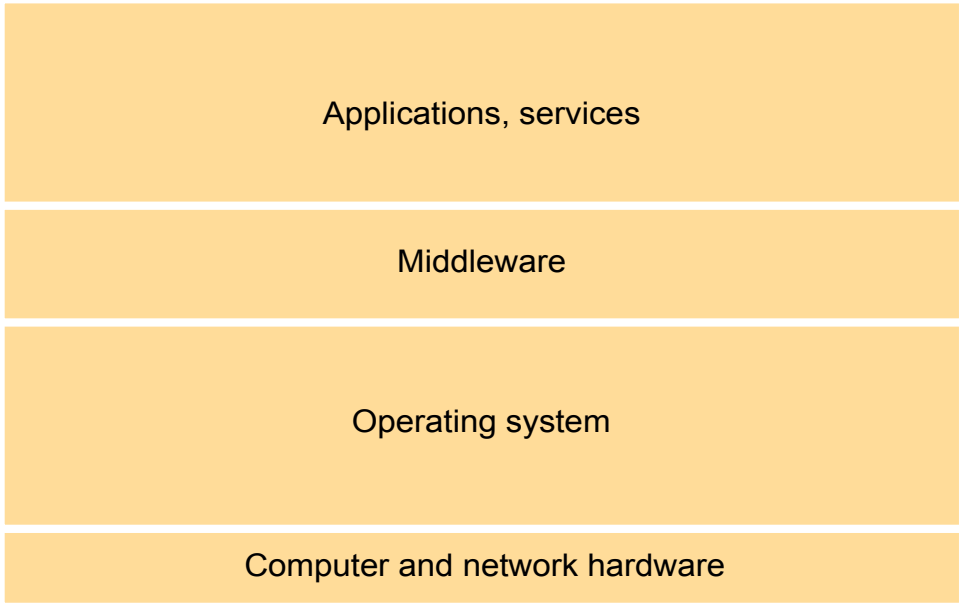








Middleware



Source: Colouris

- **Platform:** underlying hardware and operating systems.
- **Middleware:** common communication and support services.
- **Applications:** software built on top of those services.

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Middleware
Increases developer productivity

The main goal of middleware is to isolate the developer from the non-functional features of his application/service.

- Ideally, problems with resource location, failures, performance, scalability, etc. should be transparent to the developer.

From the developer's point of view:

- Isolate yourself from the SD particularities
- Make your development transparent
- Focus on functional requirements
- Non-functional requirements should be provided by the platform

The term *middleware* is coined.

Middleware
How to achieve homogeneity

Universal references

- A unique non-ambiguous way to locate resources.
- Resource Naming.

Canonical representation

- Common network-level data representation and format.

Language bindings

- Definition of common interfaces.
  - The contract between peers.
- How to map representations to different programming languages.

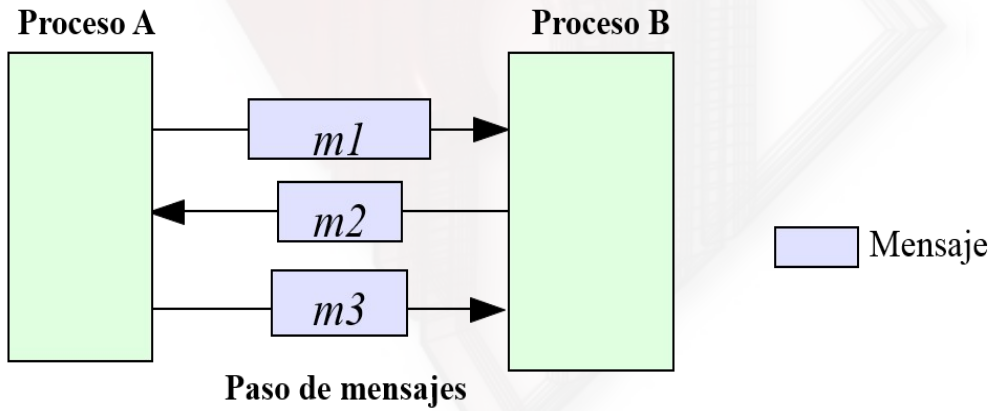
Message passing

Fundamental model of distributed applications:

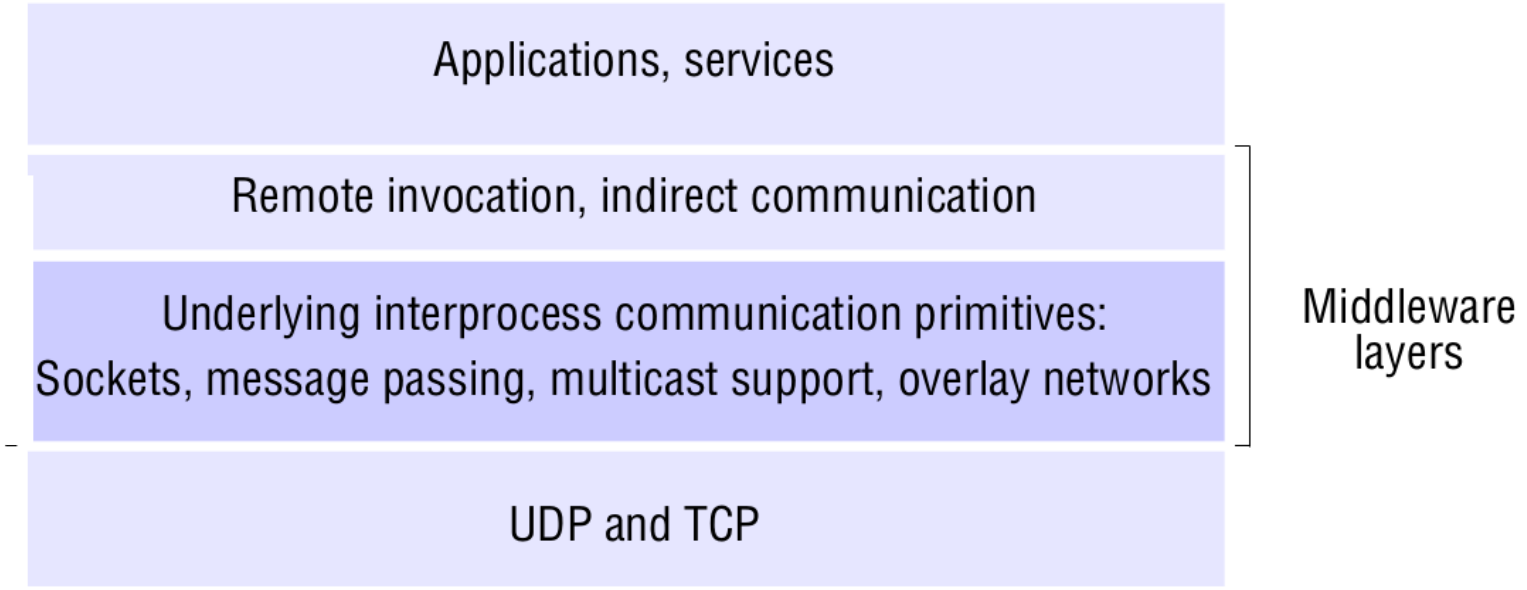
- A sending process sends a request message.
- The message arrives at the receiving process, which processes the request and returns a reply message.
- This reply can generate further requests from the sending process.

Services exchange messages defined for each specific application.

- Data Oriented
- Each message must be interpreted
- Coupled services



Distributed Computing paradigms



Source: Colouris

- Sockets
- Client/Server
- Remote invocation (RPC / RMI)
- Indirect communication

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Sockets

IPC mechanism (ie, provides communication between processes) eventually running on different computers

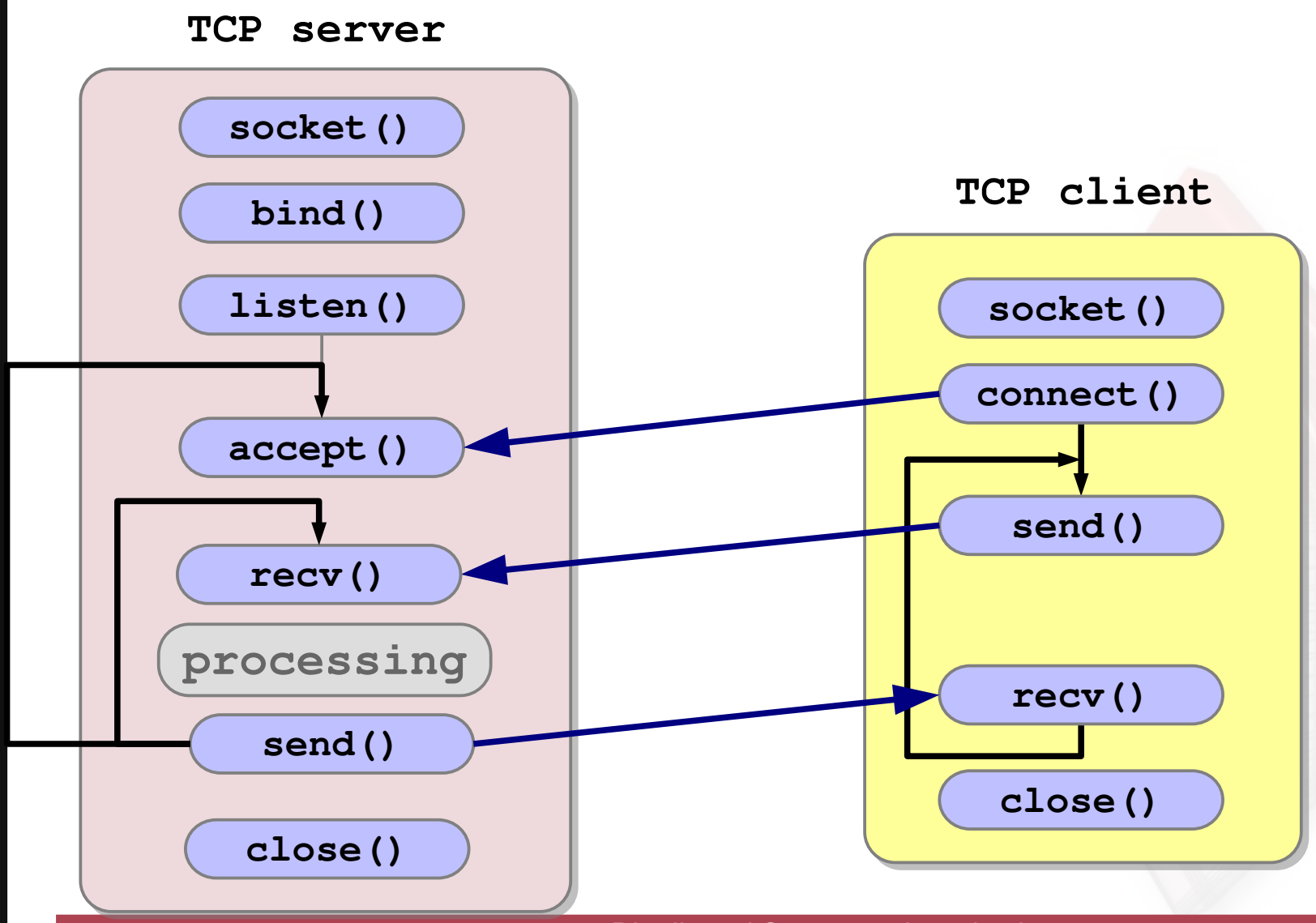
- Virtually universal low-level interface
- Access to link, network and transport layers
- Very efficient, but complex and error-prone
- Protocol agnostic
- Distributed application features are implemented ad-hoc
  - A protocol is designed for each application.

UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Client/Server

- Assigns different roles to the processes that communicate: client and server
  - Client: request a service from the server
    - Active entity: invokes requests
  - Server: offers a service
    - Manages resources (hardware, software, data)
    - Passive entity: wait for incoming requests
- Clients and servers can be objects or processes running on different computers
- Examples:
  - Internet services (HTTP, FTP, DNS, etc.)

Client/Server with sockets
Example: TCP server

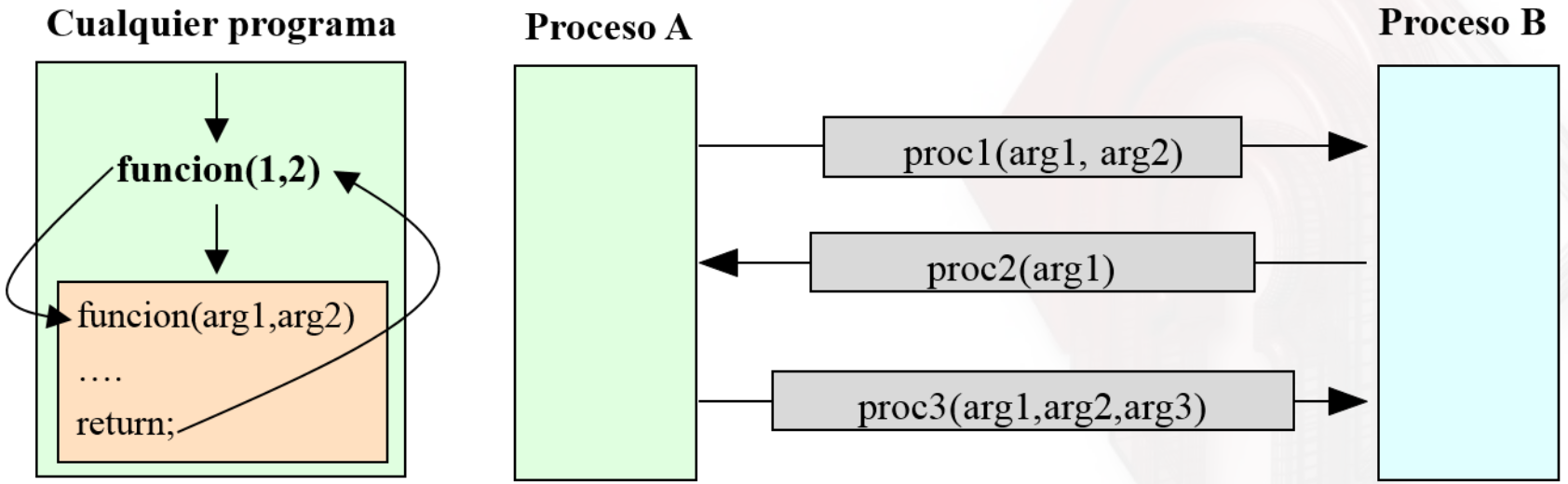


Transparency?
Reliability?
Scalability?

RPC: Remote Procedure Call

Idea: Make distributed software programming the same way a conventional (non-distributed) application.

- Conceptually the same as invoking a local (same node) procedure



Indirect Communication

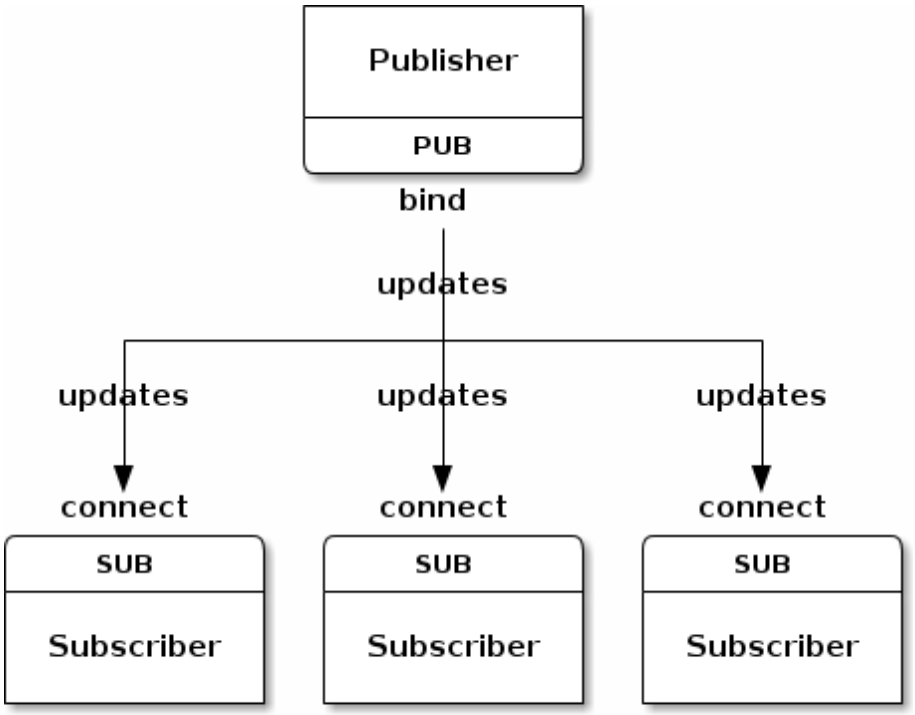


Figure 4 – Publish-Subscribe

- pull/push model
- Event channel management
- Filtering: source/destination, type, content, etc.

References

G. Coulouris, *Distributed Systems: Concepts and Design*, Addison Wesley 2011

- Chapter 1 – Characterization of distributed systems.
- Chapter 2 – System models

Credits

Authors

- David Villa Alises
- Félix Jesús Villanueva Molina
- Soledad Escolar Díaz
- Ana Rubio Ruiz

Contributors

- José Antonio de la Torre las Heras